QML Lecture · 3일차

실습 설명 자료

MediQ-GAN 미세조정 → QGAN과 같은 지표로 비교

오리엔테이션

3일차 진행 순서

노트북 섹션 순서대로 진행하고, 섹션 앞에서 해당 PART를 설명함

노트북	내용	실행 시간	설명 자료
§ 0 – § 1	Colab 설정, 저장소 clone, 실습 자산 로드	1분 이내	PART 0
§ 2	QGAN checkpoint(600 step) 생성·평가 - 비교 기준선	수 초	PART 0
§ 3	MediQ-GAN 구조: dual-stream, 16큐빗 회로 × 5	십여 초	PART 1
§ 4	**10 epoch checkpoint에서 20 step 미세조정**	**약 7분 (로컬 CPU)**	**PART 2**
§ 5	QGAN과 MediQ-GAN을 같은 지표 4종으로 비교	수 초	PART 3
§ 6	이미지 생성에서 양자컴퓨팅의 한계와 방향	-	PART 4

§ 4 셀은 먼저 실행해 두고, 학습이 진행되는 동안 PART 2를 설명함.

PART 0 · 노트북 §1 – §2

2일차 복습과 QGAN 기준선

2일차에 만든 QGAN을 checkpoint로 불러와 오늘 비교의 기준선으로 씀

노트북 범위	§1 실습 자산 로드, §2 QGAN 생성·평가
다루는 코드	torch.load · PatchQuantumGenerator(**config) · gm.evaluate

PART 0 · 2일차 복습

2일차 구성과 3일차에서 바뀌는 부분

생성자만 MediQ-GAN으로 바꾸고, 데이터·판별자·지표는 2일차 그대로 씀

항목	2일차 (QGAN)	3일차 (MediQ-GAN)
데이터	DF 115장, segmentation 후 16×16, 0 ~ 1	2일차와 같음
생성자	**sub-generator 4개 × 7큐빗, depth 10 - 생성자 전체가 양자회로**	**고전 stream + 양자 stream(16큐빗 × 5, depth 8)**
출력 척도	측정 확률 (patch별 합 0 ~ 1)	Tanh (-1 ~ 1)
판별자	5층 FC (논문 Table 4)	2일차와 같음
지표	상관 · 최근접 거리 · 다양성 · 픽셀 분포	2일차와 같음
학습량	수업 중 400 step, 강사 checkpoint 600 step	사전학습 25 epoch, 수업 중 20 step 미세조정

checkpoint 불러오기

config로 같은 구조를 먼저 만들고, 학습된 가중치를 덮어씀

```
ckpt_path = f"{ASSET_DIR}/qgan_df_{IMG_SIZE}px.pt"
checkpoint = torch.load(ckpt_path, map_location="cpu")
cfg = checkpoint["config"]

gen = PatchQuantumGenerator(
    n_generators=cfg["n_generators"],
    n_data_qubits=cfg["n_data_qubits"],
    n_ancillas=cfg["n_ancillas"], q_depth=cfg["q_depth"])
gen.load_state_dict(checkpoint["generator_state_dict"])
gen.eval()
```

checkpoint 키	내용
generator_state_dict	생성자 가중치 (회전각 등)
discriminator_state_dict	판별자 가중치
config	모델 재구성에 필요한 설정
history	step별 loss_g · loss_d
epoch	학습한 epoch 수 (MediQ-GAN)

이 파일의 config: sub-generator 4, 데이터 6 + ancilla 1, depth 10, 600 step

QGAN 기준선

강사 checkpoint(600 step)를 지표 4종으로 평가한 값. PART 3에서 MediQ-GAN과 같은 표에 놓음

지표	QGAN 600 step	비교 기준	해석
② 클래스 평균 상관 ↑	**0.195 ± 0.012**	높을수록 DF 평균 형태에 가까움	64장 × 10시드 평균 ± 표준편차
③ 최근접 실제 거리	**1.64**	0에 가까우면 학습 이미지 암기	암기는 아님
④ 샘플 간 거리	**1.92**	실제 DF 2.505	실제의 약 77%

```
with torch.no_grad():
    qgan_fake = gen(batch_size=64).numpy().reshape(64, 16, 16)
qgan_scores = gm.evaluate(qgan_fake, df_imgs, "prob")
```

생성할 때마다 새 난수를 쓰므로 노트북 실행 결과는 표와 조금씩 다름. 소수점 자리보다 모델 간 격차를 봄.

수치: README 결과표 (로컬 CPU, PennyLane exact 시뮬레이션)

PART 1 · 노트북 §3 직전

MediQ-GAN 구조

생성자 안에서 고전 stream과 양자 stream이 병렬로 돌고 하나로 합쳐짐

노트북 범위	§3 ToyDualStreamGenerator 정의와 forward 1회
다루는 코드	q_circuit · quantum_stream · forward
비교 대상	2일차 QGAN(생성자 전체가 양자회로)

hybrid 수준 : model-level과 layer-level

QGAN은 생성자 전체가 양자회로, MediQ-GAN은 생성자 안의 한 갈래만 양자회로

QGAN - model-level hybrid

Andra et al., 2025 · 2일차 실습

- 생성자 = 양자회로 (sub-generator 4개)
- patch를 나눠 만들고 이어붙임
- 출력 = 측정 확률 (patch별 합 0 ~ 1)
- 판별자만 고전

z → [양자회로 × 4] → 이미지

MediQ-GAN - layer-level hybrid

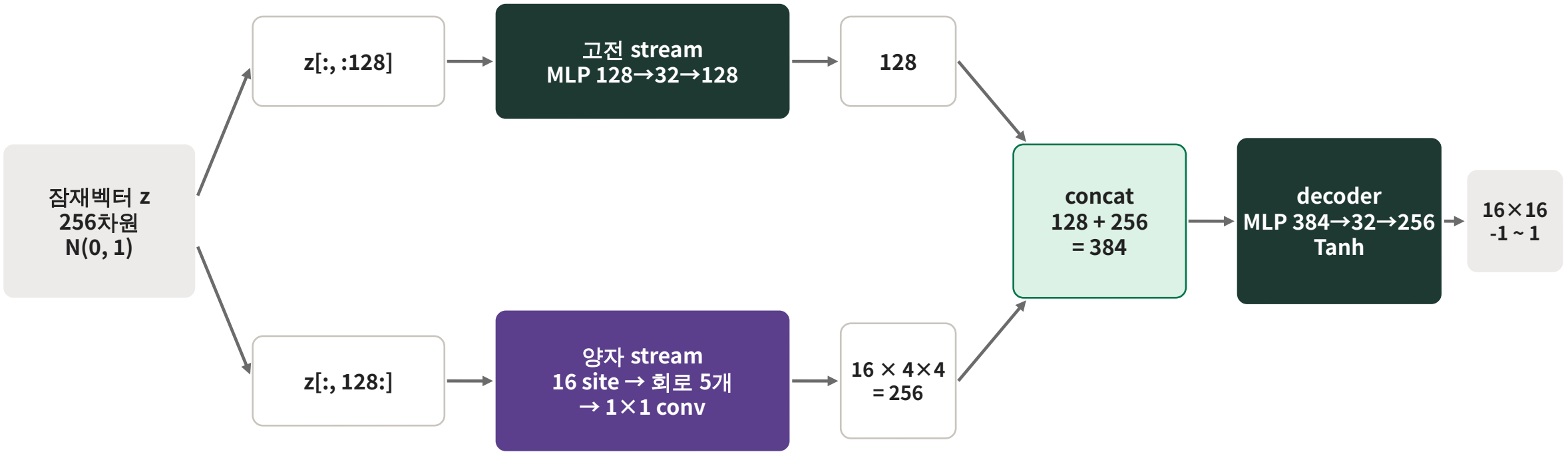
Jiao et al., 2025 · 3일차 실습

- 생성자 = 고전 stream + 양자 stream
- 입력을 절반씩 나눠 병렬 처리 후 concat
- 출력 = 고전 decoder → Tanh (-1 ~ 1)
- 양자 stream은 원 논문 스펙(16큐빗 × 5, depth 8)

z → [고전 ‖ 양자] → concat → decoder

dual-stream의 데이터 흐름

잠재벡터 256차원을 절반씩 나누고, 두 stream의 출력을 이어붙여 decoder로 보냄

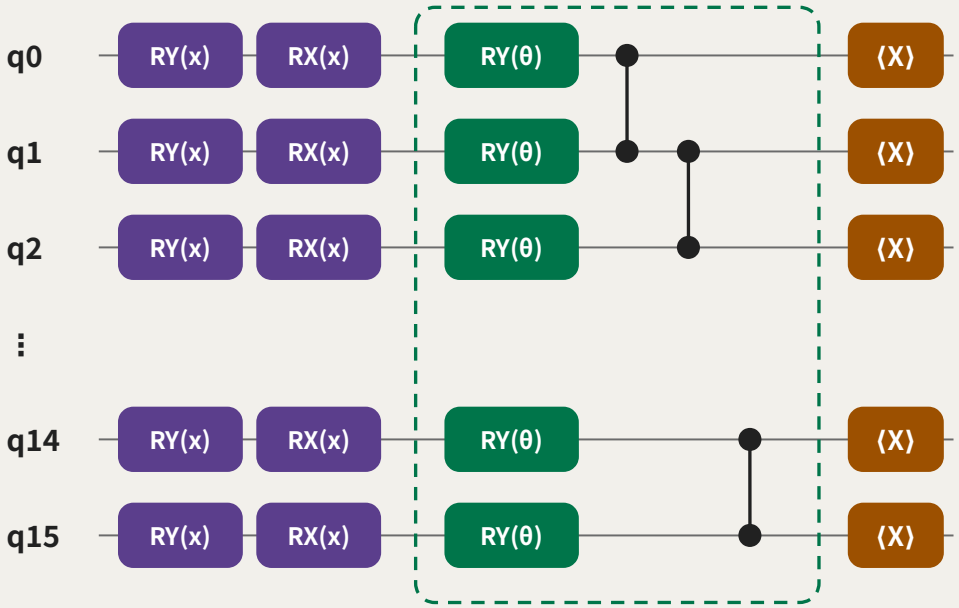



```
z_c, z_q = z[:, :self.half], z[:, self.half:]
fused = torch.cat([self.classical_stream(z_c),
                   self.quantum_stream(z_q)], dim=1)
return self.decoder(fused)
```

양자 회로 : 16큐빗, depth 8

같은 값으로 RY → RX encoding, [학습 RY + 인접 CZ] × 8, 큐빗별 PauliX 기댓값

```
@qml.qnode(dev, interface="torch", diff_method="adjoint")
def q_circuit(x, weights):
    for i in range(n_qubits):          # 1 encoding
        qml.RY(x[i], wires=i)
        qml.RX(x[i], wires=i)
    for layer in range(q_depth):       # 2 x 8
        for i in range(n_qubits):
            qml.RY(weights[layer, i], wires=i)
        for i in range(n_qubits - 1):
            qml.CZ(wires=[i, i + 1])
    return [qml.expval(qml.PauliX(i))  # 3
            for i in range(n_qubits)]
```



출력: 큐빗 16개의 기댓값 → 평균 → site당 스칼라 1개

양자 stream : 16 site 중 5개만 회로로 처리

양자 쪽 입력을 4×4 = 16 site로 보고, 고른 5개 site만 sub-generator 회로가 처리함

site 0 gen 1	site 1 0	site 2 0	site 3 0
site 4 gen 2	site 5 0	site 6 0	site 7 0
site 8 gen 3	site 9 0	site 10 0	**site 11 gen 4**
site 12 0	site 13 0	site 14 0	**site 15 gen 5**

4×4 site 맵 - 보라색 5칸만 회로 출력, 나머지는 0

```
sites  = z_quantum.view(B, 16, self.site_dim)   # (B,16,8)
tokens = self.q_align(sites)                    # (B,16,16)
q_maps = torch.zeros(B, 5, 4, 4)
for g, si in enumerate(self.site_idx):  # [0,4,8,11,15]
    outs = torch.stack([torch.stack(self.q_circuit(
                tokens[b, si], self.q_weights[g]))
            for b in range(B)])           # (B, 16) expvals
    y, x = divmod(si, 4)
    q_maps[:, g, y, x] = outs.mean(dim=1)  # scalar
return self.proj_qexp(q_maps).flatten(1)   # (B, 256)
```

forward 1회의 회로 실행 수 = 5 × batch. 회로 하나의 상태벡터는 2^{16} = 65,536차원.

생성자 파라미터 구성

30,000개 중 양자 회전각은 640개(2.1%). 2일차 QGAN 생성자는 280개 전부가 양자 회전각임

모듈	구성	파라미터	비율
decoder	Linear 384 → 32 → 256 + Tanh	20,768	69.2%
고전 stream	Linear 128 → 32 → 128	8,352	27.8%
양자 회전각	**sub-generator 5 × 8층 × 16큐빗**	**640**	**2.1%**
q_align	Linear 8 → 16 (site 벡터를 큐빗 수에 맞춤)	144	0.5%
proj_qexp	1×1 conv, 5 → 16채널	96	0.3%
합계		**30,000**	100%

파라미터 수로는 2%지만, 학습 시간의 대부분은 이 640개가 들어 있는 16큐빗 회로의 시뮬레이션에서 나옴.

노트북 ToyDualStreamGenerator(in_dim 256)에서 직접 센 값

PART 1 · 한계

원 논문과 실습의 차이

양자 stream은 원 논문 스펙 그대로, 고전 부분과 판별자·평가를 단순화함

항목	원 논문 (Jiao et al.)	3일차 실습	비고
양자 stream	16큐빗 × 5, depth 8	**동일**	공식 레포와 같은 회로
고전 encoder / decoder	conv	MLP	16×16 저해상도라 단순화
입력 · 출력	64×64 RGB	16×16 그레이스케일	2일차 전처리 자산 사용
판별자	WGAN-GP critic	표준 BCE 판별자 (Table 4)	QGAN과 같은 판별자
학습	GPU, 전체 학습	25 epoch 사전학습 + 수업 중 미세조정	16큐빗 시뮬레이션 비용
평가	FID, 분류 성능	지표 4종	FID는 16×16에서 의미 없음

PART 2 · 노트북 §4 셀 실행 후

라이브 미세조정

덜 학습된 10 epoch checkpoint에서 출발해 20 step을 더 학습함

노트북 범위	§4 미세조정과 전후 비교
다루는 코드	mediqgan_e010.pt · sample_z · real_tanh · Adam(betas=(0.5, 0.999))
소요 시간	20 step 약 7분 (로컬 CPU), Colab은 더 걸림

PART 2 · 노트북 §4 코드

미세조정 코드와 2일차 설정 비교

루프 구조는 2일차와 같고, 출력 척도·잠재벡터·optimizer 설정이 다름

```
mq_ckpt = torch.load(f"{ASSET_DIR}/mediqgan_e010.pt")
mq_disc.load_state_dict(mq_ckpt["discriminator_state_dict"])

real_tanh = torch.tensor(df_imgs.reshape(115, -1) * 2 - 1)
def sample_z(n): return torch.randn(n, 256)   # z ~ N(0, 1)

opt_g = Adam(mq_gen.parameters(),  lr=2e-4, betas=(0.5, 0.999))
opt_d = Adam(mq_disc.parameters(), lr=2e-4, betas=(0.5, 0.999))

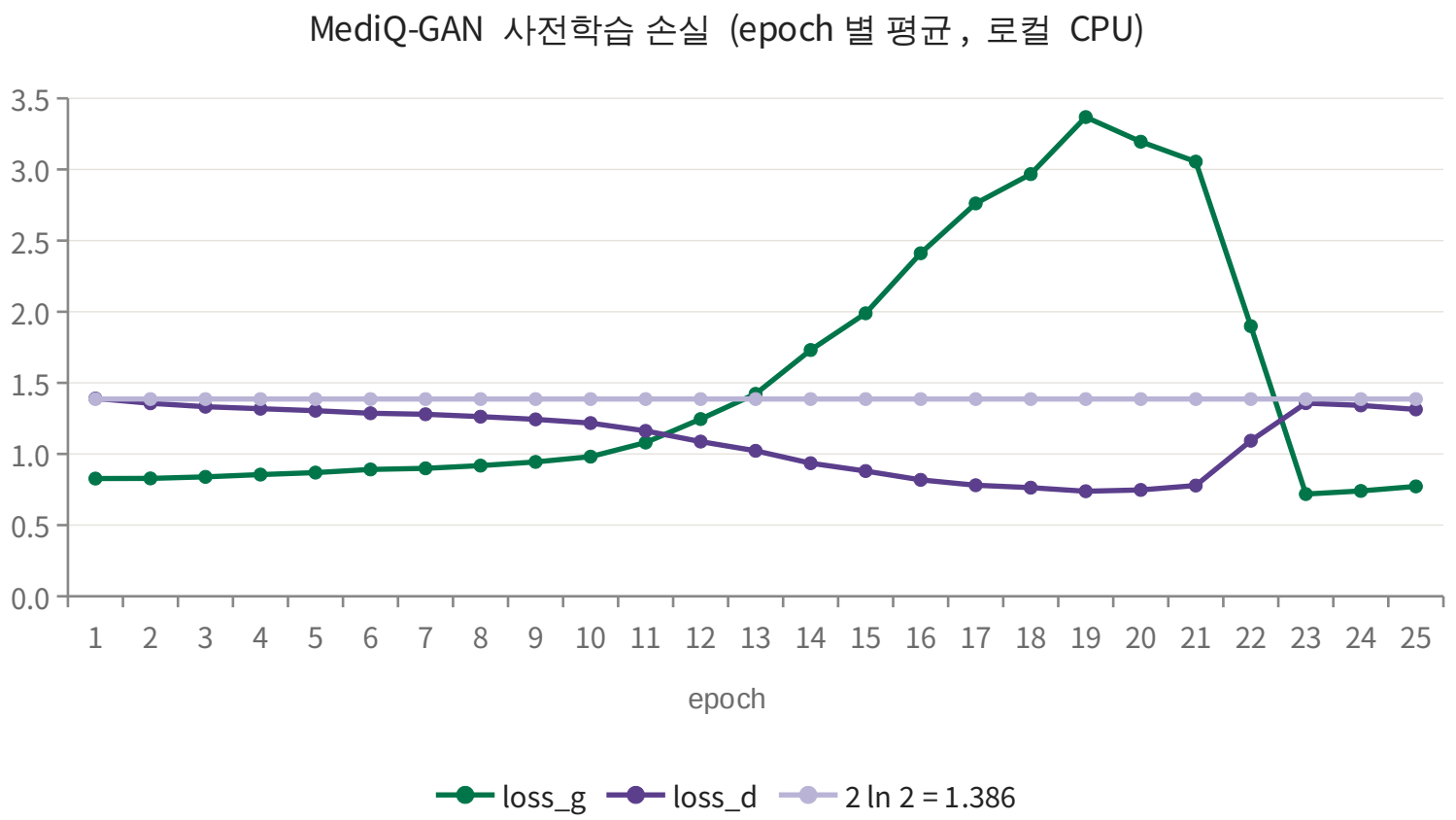
for step in range(20):       # batch 4, same D/G loop as day 2
    ...
```

설정	2일차 QGAN	3일차 MediQ
출력 척도	확률 (prob)	Tanh (-1 ~ 1)
실제 이미지	이미지 합 = 1	2x - 1
잠재벡터	U(0, π) 각도	N(0, 1)
학습률 G / D	0.05 / 0.001	2e-4 / 2e-4
Adam betas	(0.9, 0.999)	(0.5, 0.999)
batch · step	8 · 400	4 · 20 (미세조정)

lr 2e-4, beta1 0.5는 DCGAN 논문(Radford et al., 2016)의 설정값

사전학습 손실 기록 (25 epoch)

강사 checkpoint의 학습 기록(epoch별 평균). 15 ~ 21 epoch에서 판별자가 우세해졌다가 22 epoch부터 회복함



10 epoch까지

loss_d 1.39 → 1.22, 2 ln 2 근처의 균형 상태

11 ~ 21 epoch

loss_d 최저 0.74 (19 epoch), loss_g 최고 3.37 - 판별자 우세

22 epoch 이후

loss_d 1.3 부근으로 회복

lr_g 0.02 사전 실험

3 epoch 만에 loss_d 0.0027, 다양성 0.000 (완전 mode collapse)

mediqgan_final.pt history, 700 step = 25 epoch × 28 step

계산 비용 : 수업에서 미세조정만 하는 이유

16큐빗 회로 시뮬레이션이 비싸 25 epoch 사전학습에 196분(약 3시간 15분)이 걸림

	2일차 QGAN	3일차 MediQ-GAN
큐빗 (회로 하나)	7	16
상태벡터 차원	$2^7 = 128$	$2^{16} = 65{,}536$
미분 방식	backprop (default.qubit)	adjoint (lightning.qubit)
1 step	약 2.2초 (batch 8)	**약 20초 (batch 4)**
전체 학습	600 step 약 22분	**25 epoch 196분 (epoch당 7.8분)**
수업 중 학습	처음부터 400 step (Colab 약 35분)	10 epoch checkpoint에서 20 step

16큐빗은 12큐빗보다 상태벡터가 16배 크고(2^{16} vs 2^{12}), 실측 소요 시간은 약 20배였음.

로컬 CPU 실측. epoch 시간은 checkpoint epoch_sec 평균

PART 3 · 노트북 §5 직전

QGAN vs MediQ-GAN

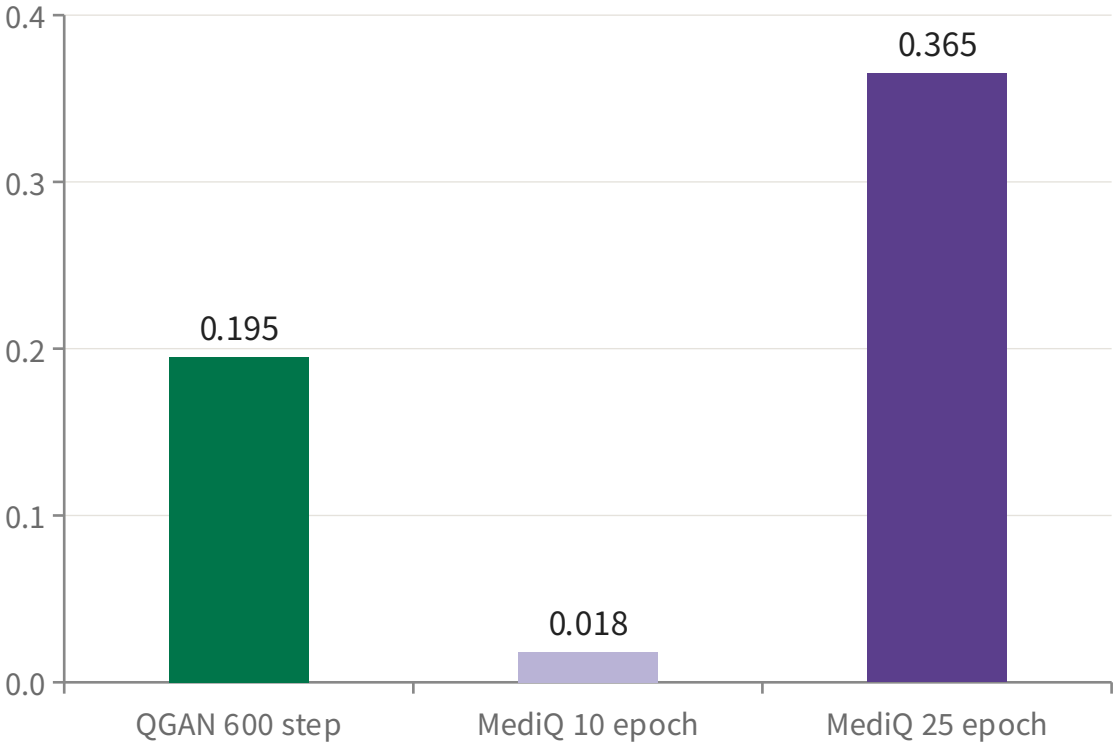
두 모델의 출력을 0~1로 되돌린 뒤 같은 지표 4종으로 비교함

노트북 범위	§5 지표 비교표, 상관계수·픽셀 분포 그림
다루는 코드	gm.comparison_table · to_unit_scale(x, 'prob' / 'tanh')

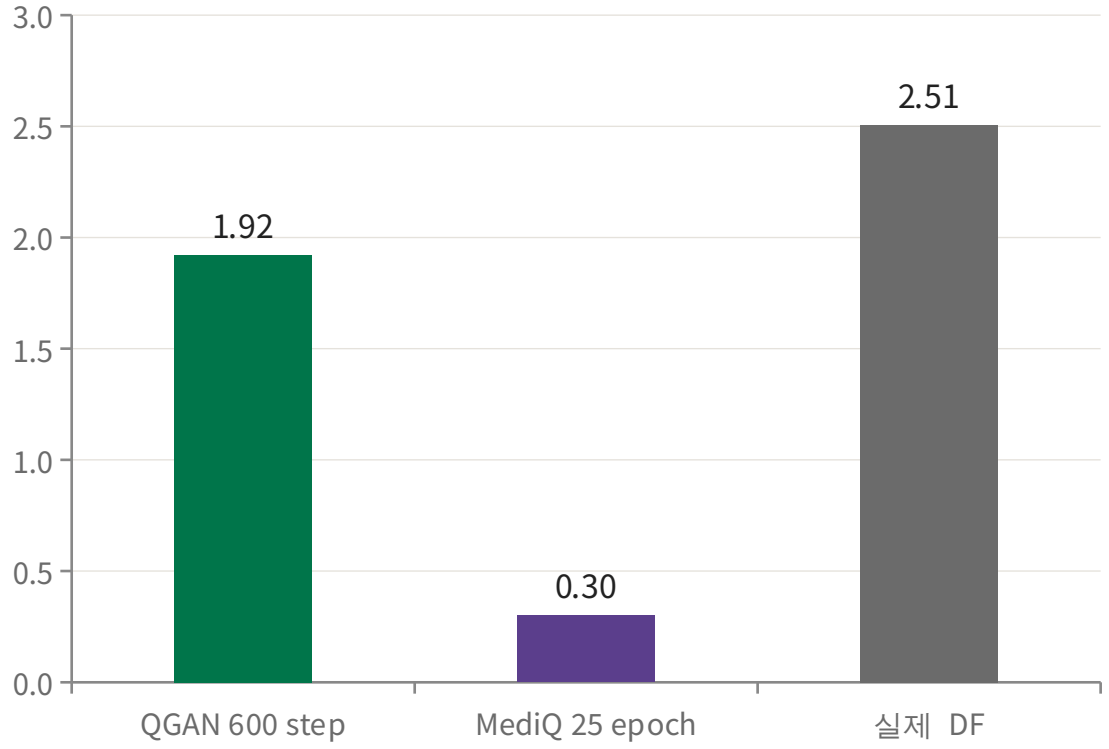
지표 비교 : 형태 재현과 다양성

상관은 MediQ-GAN이 높고, 샘플 간 거리는 QGAN이 실제 DF에 가까움

② 클래스 평균 상관 (높을수록 형태 재현)



④ 샘플 간 거리 (0 에 가까우면 mode collapse)



수치: README 결과표 (로컬 CPU). MediQ-GAN은 16장, QGAN은 64장으로 평가

결과 해석 : 두 모델의 한계

QGAN은 형태 재현이 약하고, MediQ-GAN은 평균 이미지에 가까운 그림을 반복함

지표	QGAN 600 step	MediQ-GAN 25 epoch	의미
② 클래스 평균 상관 ↑	0.195	**0.365**	평균 형태는 MediQ-GAN이 더 가까움
③ 최근접 실제 거리	1.64	3.44	MediQ-GAN은 실제 이미지 어느 것과도 멂
④ 샘플 간 거리 (실제 2.505)	**1.92**	0.30	MediQ-GAN은 비슷한 그림을 반복

해석

QGAN: 다양성은 실제에 가깝지만 병변 형태 재현이 약함. patch를 서로 독립적으로 만들어 이미지 전체 구조를 잡기 어려움.
MediQ-GAN: 평균 형태는 잘 맞추지만 다양성이 실제의 1/8. 최종 출력을 고전 decoder가 만들기 때문에 Tanh가 포화되면 z와 무관한 이미지가 나옴.
②만 보면 MediQ-GAN이 낫다고 결론 내리게 됨. ②는 평균 형태만 보므로 같은 이미지를 반복해도 높게 나옴.

PART 4 · 노트북 §6

이미지 생성에서 양자컴퓨팅의 한계

3일 동안의 실습에서 확인한 내용 (노트북 §6)

	관찰	실습에서 확인한 근거
①	측정 방식이 학습 성패를 가름	ancilla qubit = 0만 쓰도록 바꾸자 생성 평균과 실제 평균의 상관 0.00 → 0.49 (8×8 사전 실험)
②	현재 학습은 시뮬레이터에서만 가능한 규모	두 모델 모두 exact 상태벡터 시뮬레이션. 실제 장치는 parameter-shift로 파라미터당 회로 2회 × shot
③	해상도 한계가 평가 한계로 이어짐	16×16에서는 FID를 쓸 수 없어 지표 4종으로 대신함
④	큐빗 수에 지수적으로 늘어나는 비용	16큐빗은 12큐빗 대비 상태벡터 16배, 실측 시간 약 20배
⑤	고전 쪽 학습 안정화가 결과를 좌우	BCE 판별자가 쉽게 이김. 원 논문은 WGAN-GP critic으로 판별자를 제약
⑥	고전 이미지 생성에서의 우위는 미검증	양자 생성모델은 양자 데이터·해밀토니안 상태 생성에서 먼저 의미를 가짐

PART 4 · 참고

MediQ-GAN 원 논문 결과

논문 수치 인용 - 실습의 16×16 결과와 직접 비교하지 않음

데이터셋	내용	결과
ISIC 2019	피부경검사 8클래스	소수클래스(df, vasc, scc)에서 모든 고전 baseline보다 낮은 FID
ODIR-5k	안저사진	증강 후 분류: 두 backbone(EfficientNet-B0, ViT-small) 모두 최고 성능
RetinaMNIST	다운샘플 안저사진	5개 클래스 중 4개에서 최저(최고품질) FID

비교 대상 DCGAN, WGAN-GP, Structure-Preserving GAN, StyleGAN2-ADA, FastGAN, DiT · Stable Diffusion 3(LoRA 미세조정), 이전 세대 hybrid 모델 MosaiQ

실기기 검증 IBM 양자 장치(ibm_marrakesh)에서 대표 클래스의 추론만 검증 (대기열·shot 제한)

출처: Jiao et al., MediQ-GAN, arXiv:2506.21015v2, Table 1 · 2

확인 질문

답은 발표자 노트에 있음

1. QGAN과 MediQ-GAN의 hybrid 수준 차이를 '어디까지가 양자인가'로 설명하면
2. checkpoint에 가중치뿐 아니라 config를 함께 저장하는 이유는
3. MediQ-GAN 양자 회로에 ancilla qubit과 post-selection이 필요 없는 이유는
4. MediQ-GAN의 상관계수가 더 높은데도 더 좋은 생성모델이라고 단정할 수 없는 이유는
5. 학습률을 크게 두거나 BCE 판별자를 쓰면 어떤 문제가 생기고, 원 논문은 어떻게 대응했는가

정리

3일 과정 정리

같은 데이터·같은 지표로 두 hybrid 전략을 나란히 비교함

1일차	데이터셋 · 두 논문 아키텍처 · VQC 학습 방식과 계산 비용
2일차	segmentation 전처리 · sub-generator 회로 · QGAN 학습 · 지표 4종
3일차	MediQ-GAN 구조 · 미세조정 · QGAN과 같은 지표로 비교
결론	형태(상관)와 다양성을 함께 봄 · 양자 비용과 학습 안정화가 결과를 좌우함